

Grafer i C#: BFS og DFS

Toveis forbindelser · Korteste vei · Finnes det en rute? · Syklus i urettet graf

Slik bruker du opplegget

Du kan allerede traversere en graf, så vi hopper over grunnkurset og går rett på det som skiller de fire oppgavene fra vanlig traversering. Hver del følger samme oppskrift:

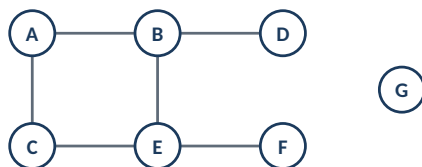
1. **Idé** – den ene tingen som gjør oppgaven annerledes enn ren traversering.
2. **Kode** – kort, komplett og skrevet for å pugges. Samme navn og struktur hele veien.
3. **Sporing** – vi kjører algoritmen for hånd på eksempelgrafene under.
4. **Puggregel og vanlige feil** – det du skal kunne uten å tenke.

Til slutt får du et puggark, en fyll-inn-test, kontrollspørsmål med fasit og et komplett program du kan kjøre. Anbefalt arbeidsmåte: les en del, lukk dokumentet, skriv koden fra hukommelsen, sammenlign.

Del 1	Grafen og toveis forbindelser
Del 2	BFS: korteste vei mellom to noder
Del 3	DFS: finnes det en rute?
Del 4	DFS: syklus i urettet graf
Del 5	BFS og DFS side om side
Del 6	Puggark
Del 7	Fyll inn hullene
Del 8	Test deg selv, med fasit
Vedlegg	Komplett program (Program.cs)

Eksempelgrafene

Alle eksemplene bruker denne grafen. Lær deg den, så kan du spore algoritmene i hodet.



Kanter: A-B, A-C, B-D, B-E, C-E, E-F, C-D, E-G. G er en isolert node uten naboer.

Node	Naboliste (i innsettingsrekkefølge)
A	B, C
B	A, D, E
C	A, E
D	B
E	B, C, F
F	E
G	(tom)

Rekkefølgen i listene er den rekkefølgen kantene ble lagt til i. Den bestemmer hvilken vei algoritmene prøver først, og dermed hvilken av flere like gode løsninger du får.

Del 1 Grafen og toveis forbindelser

Representasjon: nabolister

En graf lagres enklest som en `Dictionary<string, List<string>>`: nøkkelen er noden, verdien er listen over naboene. Oppslag på naboer er $O(1)$, og det er trivielt å legge til noder og kanter underveis. Nodene identifiseres med `string`, slik oppgaven krever:

```
AddConnection(string nodeA, string nodeB).
```

Idé: en toveis forbindelse er to enveis kanter

I en urettet graf betyr kanten A–B både «A kan gå til B» og «B kan gå til A». Men nabolisten er i seg selv rettet: `_adj[A]` sier bare hva A kan nå. Skal kanten være toveis, må du legge B inn i A sin liste **og** A inn i B sin liste. Det er hele trikset: to `Add`-kall i stedet for ett.

```
public class Graph
{
    // Nabolister: hver node -> listen over naboene sine
    private readonly Dictionary<string, List<string>> _adj = new();

    public IEnumerable<string> Nodes => _adj.Keys;

    public void AddNode(string node)
    {
        _adj.TryAdd(node, new List<string>()); // ingen effekt hvis noden finnes fra før
    }

    public void AddConnection(string nodeA, string nodeB)
    {
        AddNode(nodeA); // sørg for at begge nøklene finnes
        AddNode(nodeB);
        _adj[nodeA].Add(nodeB); // nodeA -> nodeB
        _adj[nodeB].Add(nodeA); // nodeB -> nodeA <-- toveis
    }

    public IReadOnlyList<string> Neighbors(string node)
    {
        if (_adj.TryGetValue(node, out var list))
            return list;
        return Array.Empty<string>(); // ukjent node: tom liste, ikke exception
    }
}
```

Tre detaljer som er verdt å merke seg

- `AddNode` bruker `TryAdd`, som ikke gjør noe hvis nøkkelen finnes fra før. Derfor kan `AddConnection` trygt kalle den for begge nodene hver gang, uten å overskrive en eksisterende naboliste.
- Isolerte noder som G må legges til med `AddNode` direkte. Ellers finnes de ikke i grafen.
- `Neighbors` bruker `TryGetValue` og gir en tom liste for ukjente noder. Da får du aldri `KeyNotFoundException` midt i en traversering, selv om noen spør etter en node som aldri ble lagt inn.

Vanlige feil

- **Glemt `_adj[nodeB].Add(nodeA)`.** Grafen blir rettet uten at du merker det. Fra A finner du fortsatt D, men fra D finner du ingenting – D sin liste er tom.
- **Glemt `AddNode` inne i `AddConnection`.** `_adj[nodeA].Add(nodeB)` kaster `KeyNotFoundException` første gang `nodeA` er ny.
- **Doble forbindelser.** `AddConnection("A", "B")` to ganger gir B to ganger i A sin liste. Traversering tåler det (besøkt-settet stopper dobbeltbesøk), men syklusdeteksjonen i del 4 vil melde syklus – to parallelle kanter er teknisk sett en syklus. Vil du forby duplikater, bytt `List<string>` med `HashSet<string>`.

Puggregel: toveis forbindelse

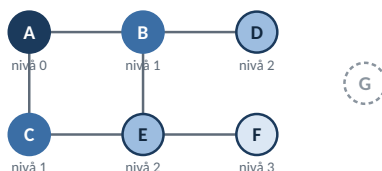
`AddConnection = AddNode(nodeA), AddNode(nodeB), nodeA → nodeB, nodeB → nodeA`. Fire linjer, alltid i den rekkefølgen.

Del 2 BFS: korteste vei mellom to noder

Kort repetisjon. BFS bruker en kø og et besøkt-sett. Den tar ut fremst i køen, legger alle ubesøkte naboer bakerst, og fortsetter til køen er tom.

Hvorfor BFS gir korteste vei

Fordi køen er FIFO, behandler BFS nodene i økende avstand fra start: først alle med avstand 1, så alle med avstand 2, og så videre. Første gang en node blir oppdaget, er den derfor oppdaget via en av de korteste veiene – ingen senere oppdagelse kan være kortere. Det er hele beviset. Forutsetningen er at alle kanter koster like mye, altså en uvektet graf.



Nivåene sett fra A. BFS tømmer ett nivå før den begynner på neste. G nås aldri.

Idé: husk hvem som oppdaget hvem

Vanlig BFS forteller deg bare *at* du kom fram, ikke *hvordan*. Løsningen er én ekstra datastruktur: en forelder-tabell `parent`, der `parent[x]` er noden som oppdaget x. Den fylles ut med én linje i det øyeblikket en nabo legges i køen. Når du tar målet ut av køen, går du baklengs fra mål til start via `parent`, og snur listen.

```
public List<string>? ShortestPath(string start, string goal)
{
    var queue = new Queue<string>();
    var visited = new HashSet<string> { start };
    var parent = new Dictionary<string, string>(); // barn -> forelder (hvem oppdaget hvem)

    queue.Enqueue(start);

    while (queue.Count > 0)
    {
        var current = queue.Dequeue();

        if (current.Equals(goal))
            return BuildPath(parent, start, goal);

        foreach (var neighbor in Neighbors(current))
        {
            if (visited.Add(neighbor)) // Add returnerer true bare første gang
            {
                parent[neighbor] = current; // <-- den ene ekstra linjen
                queue.Enqueue(neighbor);
            }
        }
    }

    return null; // goal kan ikke nås fra start
}
```

Stien bygges av `parent`-tabellen, baklengs fra målet:

```
private static List<string> BuildPath(Dictionary<string, string> parent, string start, string goal)
{
    var path = new List<string>();
    var node = goal;

    while (!node.Equals(start)) // gå baklengs: mål -> ... -> start
    {
        path.Add(node);
        node = parent[node];
    }
    path.Add(start);
    path.Reverse(); // snu, så stien går start -> mål
    return path;
}
```

Sporing: ShortestPath("A", "F")

Steg	Tar ut	Naboer	Nye (forelder)	Kø etterpå
1	A	B, C	B (A), C (A)	B, C
2	B	A, D, E	D (B), E (B)	C, D, E
3	C	A, E	—	D, E
4	D	B	—	E
5	E	B, C, F	F (E)	F
6	F	F er målet: bygg stien		

Forelder-tabellen til slutt: B←A, C←A, D←B, E←B, F←E. Rekonstruksjon: start i F. $F \rightarrow \text{parent}[F] = E \rightarrow \text{parent}[E] = B \rightarrow \text{parent}[B] = A$, som er start. Listen blir [F, E, B, A]; snudd: $A \rightarrow B \rightarrow E \rightarrow F$, lengde 3.

Merk

$A \rightarrow C \rightarrow E \rightarrow F$ er like kort. BFS fant veien via B fordi B står før C i A sin naboliste, så E ble oppdaget fra B først. Korteste *lengde* er entydig; korteste *vei* er det ikke nødvendigvis.

Variant: bare avstanden

Trenger du kun antall steg, kan en avstandstabell erstatte både `visited` og `parent`: en node er besøkt hvis den har fått en avstand.

```
public Dictionary<string, int> Distances(string start)
{
    var dist = new Dictionary<string, int> { [start] = 0 }; // fungerer også som visited
    var queue = new Queue<string>();
    queue.Enqueue(start);

    while (queue.Count > 0)
    {
        var current = queue.Dequeue();
        foreach (var neighbor in Neighbors(current))
        {
            if (dist.TryAdd(neighbor, dist[current] + 1)) // ny node: ett steg lenger
                queue.Enqueue(neighbor);
        }
    }
    return dist; // dist[x] = antall steg fra start; mangler x, er den ikke nåbar
}
```

Puggregel: BFS korteste vei – «kø, besøkt, forelder»

1. Start i kø og i besøkt.
2. Ta ut. Er det målet? Bygg sti.
3. For hver **ny** nabo: merk besøkt, sett forelder, legg i kø.
4. Tom kø $\rightarrow$ null.

Sti = gå fra mål til start via forelder, så `Reverse()`.

Vanlige feil

- **Merker besøkt når noden tas ut, ikke når den legges inn.** Da kan samme node havne i køen flere ganger, og `parent` blir overskrevet av en senere, og mulig lengre, vei. Merk alltid ved innlegging.
- **Glemmer `Reverse()`.** Stien kommer baklengs, fra mål til start.
- **Bytter kø med stack.** Da har du DFS, og stien er ikke lenger kortest.
- **Vektete kanter.** BFS teller antall kanter, ikke kostnad. Har kantene ulik vekt, trenger du Dijkstra.

Del 3 DFS: finnes det en rute?

Kort repetisjon. DFS går så dypt som mulig langs én vei før den rygger tilbake. Rekursivt er det naturlig: besøk noden, kall deg selv på hver ubesøkt nabo.

Idé: returverdien må bæres oppover

Rutesjekk er DFS med to tillegg: (1) sjekk om noden du står på er målet, og (2) hvis et rekursivt kall finner målet, returner `true` **umiddelbart**, uten å fortsette til neste nabo. Punkt 2 er der de fleste feilene skjer: kallet `Dfs(nabo)` returnerer `true` dypt nede, men blir ignorert fordi ingen bruker returverdien.

```
public bool HasPath(string start, string goal)
{
    return HasPathDfs(start, goal, new HashSet<string>());
}

private bool HasPathDfs(string current, string goal, HashSet<string> visited)
{
    if (current.Equals(goal))
        return true; // 1. står vi på målet?

    visited.Add(current); // 2. merk før vi går videre

    foreach (var neighbor in Neighbors(current))
    {
        if (!visited.Contains(neighbor) && HasPathDfs(neighbor, goal, visited))
            return true; // 3. funnet lenger nede -> send opp
    }

    return false; // 4. ingen av veiene førte fram
}
```

Sporing: HasPath("A", "F")

```
Dfs(A)          besøkt: {A}
nabo B: Dfs(B)   besøkt: {A,B}
nabo A: besøkt, hopp over
nabo D: Dfs(D)   besøkt: {A,B,D}
nabo B: besøkt   -> false
nabo E: Dfs(E)   besøkt: {A,B,D,E}
nabo B: besøkt
nabo C: Dfs(C)   besøkt: {A,B,D,E,C}
nabo A: besøkt
nabo E: besøkt   -> false
nabo F: Dfs(F)   F er målet -> TRUE
-> true          (E sender svaret videre opp)
-> true          (B også)
-> true
```

Den første `true` oppstår i `Dfs(F)`, og linjen `if (... && HasPathDfs(...)) return true;` sender den rett opp igjen, nivå for nivå. D og C ble undersøkt forgjeves; det er prisen for å gå dypt først.

Sporing: HasPath("A", "G")

Samme start, men ingen node returnerer `true`. Etter A, B, D, E, C og F er alle naboer besøkt, alle kall returnerer `false`, og svaret er `false`. G ble aldri rørt – den ligger i en annen komponent, og det finnes ingen kant inn til den.

Iterativ variant

Rekursjon bruker call-stacken. I en graf med titusenvís av noder på rad kan det gi `StackOverflowException`. Da bruker du en eksplisitt `Stack<string>`. Legg merke til at dette er BFS med stack i stedet for kø: samme skjelett, motsatt rekkefølge.

```

public bool HasPathIterative(string start, string goal)
{
    var stack = new Stack<string>();
    var visited = new HashSet<string>();
    stack.Push(start);

    while (stack.Count > 0)
    {
        var current = stack.Pop();
        if (current.Equals(goal)) return true;
        if (!visited.Add(current)) continue; // allerede behandlet

        foreach (var neighbor in Neighbors(current))
            if (!visited.Contains(neighbor))
                stack.Push(neighbor);
    }
    return false;
}

```

BFS eller DFS til rutesjekk?

Begge svarer riktig på «finnes det en vei?». DFS er kortest å skrive rekursivt; BFS gir deg korteste vei på kjøpet. Trenger du bare ja/nei, velg den du husker best.

Puggregel: DFS rute

1. Er jeg målet? → true.
2. Merk besøkt.
3. For hver ubesøkt nabo: hvis rekursjonen sier true → true.
4. Ellers → false.

Vanlige feil

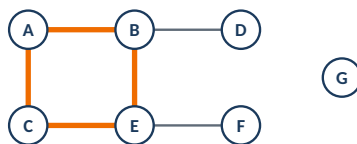
- **Ignorerer returverdien:** `HasPathDfs(neighbor, goal, visited);` alene på en linje, uten `if` og `return`. Funksjonen finner målet og glemmer det.
- **Glemmer besøkt-settet.** I en graf med syklus (A-B-E-C-A) gir det uendelig rekursjon og `StackOverflowException`.
- **Merker besøkt etter løkken i stedet for før.** Ved sykluser kan noden da bli besøkt på nytt fra en nabo før den er ferdig.

Del 4 DFS: syklus i urettet graf

Idé: en besøkt nabo som ikke er forelderen

Kjør DFS og se på naboene. En ubesøkt nabo er bare mer å utforske. Men en nabo som **allerede er besøkt** betyr at det finnes en annen vei dit enn den du står på: du har gått i ring. Det er en syklus.

Med ett unntak. I en urettet graf er kanten du kom inn på toveis, så noden du kom fra er alltid en besøkt nabo. Den teller ikke. Derfor sender vi med `parent`, altså hvor vi kom fra, og ignorerer akkurat den ene naboen.



Syklusen A-B-E-C-A. Når DFS står i C og ser at A allerede er besøkt (og A ikke er C sin forelder), er ringen sluttet.

To praktiske detaljer

- **Roten har ingen forelder.** Enkleste løsning: la roten være sin egen forelder, `CycleDfs(node, node, visited)`. Det eneste det skjuler er en kant fra roten til seg selv, som ikke finnes i en enkel graf.
- **Grafen kan ha flere komponenter.** En syklus i én komponent er usynlig fra en annen. Derfor ligger DFS-en i en ytre løkke over *alle* noder, og starter på nytt for hver node som fortsatt er ubesøkt.

```
public bool HasCycle()
{
    var visited = new HashSet<string>();

    foreach (var node in Nodes)           // ytre løkke: dekker alle komponenter
    {
        if (!visited.Contains(node) && CycleDfs(node, node, visited))
            return true;                  // roten er sin egen forelder
    }
    return false;
}

private bool CycleDfs(string current, string parent, HashSet<string> visited)
{
    visited.Add(current);

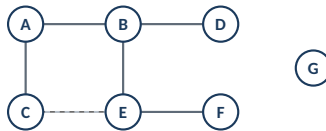
    foreach (var neighbor in Neighbors(current))
    {
        if (!visited.Contains(neighbor))
        {
            if (CycleDfs(neighbor, current, visited)) // jeg er nå forelder
                return true;
        }
        else if (!neighbor.Equals(parent))
        {
            return true; // besøkt nabo som IKKE er der vi kom fra = syklus
        }
    }
    return false;
}
```

Sporing: HasCycle() på eksempelgraf

Ytre løkke: A er ubesøkt -> CycleDfs(A, forelder=A)	besøkt: {A}
nabo B ubesøkt: CycleDfs(B, forelder=A)	besøkt: {A,B}
nabo A: besøkt, men A == forelder -> hopp over	
nabo D ubesøkt: CycleDfs(D, forelder=B)	besøkt: {A,B,D}
nabo B: besøkt, B == forelder -> hopp over	-> false
nabo E ubesøkt: CycleDfs(E, forelder=B)	besøkt: {A,B,D,E}
nabo B: besøkt, B == forelder -> hopp over	
nabo C ubesøkt: CycleDfs(C, forelder=E)	besøkt: {A,B,D,E,C}
nabo A: besøkt, og A != forelder (E) -> SYKLUS!	-> true
-> true	
-> true	
-> true	
HasCycle() = true	

C står på naboen A, som er besøkt, og A er ikke C sin forelder (det er E). Ringen er A-B-E-C-A.

Sporing: samme graf uten kanten C-E



Uten C-E er grafen en skog: to trær (A-F og G) og ingen syklus.

```
Ytre løkke: A -> CycleDfs(A, A)
  nabo B: CycleDfs(B, A)
    nabo A: == forelder, hopp over
    nabo D: CycleDfs(D, B)    nabo B == forelder    -> false
    nabo E: CycleDfs(E, B)
      nabo B: == forelder, hopp over
      nabo F: CycleDfs(F, E) nabo E == forelder    -> false
    -> false
  nabo C: CycleDfs(C, A)    nabo A == forelder    -> false
-> false
Ytre løkke: B, C, D, E, F er besøkt -> hopp over
Ytre løkke: G -> CycleDfs(G, G)    ingen naboer    -> false
HasCycle() = false
```

Hver besøkte nabo er nå forelder, og ingen kall returnerer `true`. Den ytre løkken sørger for at også G blir sjekket, selv om den ikke kan nås fra A.

Bonus: tell kantene

En sammenhengende urettet graf med V noder er et tre hvis og bare hvis den har nøyaktig $V - 1$ kanter. Generelt, med C komponenter: det finnes en syklus hvis og bare hvis $E > V - C$. Eksempelgrafen har $V = 7$, $E = 6$, $C = 2$, og $6 > 5$ stemmer: syklus. Uten C-E blir det $5 > 5$, som er usant: ingen syklus. Grei sjekk å bruke når du tester koden din.

Puggregel: DFS syklus

1. Merk besøkt.
2. For hver nabo:
 - ubesøkt $\rightarrow$ rekursjon med meg som forelder; `true` $\rightarrow$ `true`
 - besøkt og ikke forelder $\rightarrow$ `true`
3. Ellers $\rightarrow$ `false`.

Ytre løkke over alle noder. Roten er sin egen forelder.

Vanlige feil

- **Glemmer forelder-unntaket.** Første gang en node ser tilbake på den den kom fra, roper den «syklus». Enhver graf med minst én kant blir meldt som syklisk.
- **Starter bare fra én node.** Sykluser i andre komponenter blir aldri funnet.
- **Bruker `null` som forelder for roten.** Det virker for `string`, men krever ekstra null-håndtering, og faller sammen hvis du senere gjør klassen generisk. Bruk roten selv.
- **Bruker dette på en rettet graf.** Der er «besøkt nabo» ikke nok: du trenger tre tilstander (ubesøkt / under arbeid / ferdig), og forelder-unntaket skal ikke være der. Det er en annen algoritme.

Del 5 BFS og DFS side om side

	BFS	DFS
Datastruktur	Kø (FIFO)	Rekursjon eller stack (LIFO)
Rekkefølge	Nivå for nivå, utover fra start	Så dypt som mulig, så tilbake
Korteste vei (uvektet)?	Ja, garantert	Nei, bare en vei
Finnes det en rute?	Ja	Ja
Syklus i urettet graf?	Går an, men uvanlig	Ja, med forelder-unntak
Ekstra datastruktur for oppgaven	parent -tabell	parent -parameter (syklus)
Minne	Kan bli bredt: hele nivået ligger i køen	Dybden på stacken (rekursjonsdybde)
Tid, med nabolister	$O(V + E)$	$O(V + E)$
Bytt datastruktur, og du får	DFS	BFS

Fellesnevneren for alle fire oppgavene: **besøkt-settet**. Uten det går BFS i ring, DFS rekurerer uendelig, og syklusdeteksjon har ingenting å sammenligne med. V er antall noder og E antall kanter; begge algoritmene ser hver node og hver kant høyst et konstant antall ganger, derfor $O(V + E)$.

Hvilken skal du velge?

- **Korteste vei i uvektet graf** → BFS. Det er den eneste av de to som garanterer det.
- **Bare «finnes det en vei?»** → begge. DFS er kortest å skrive; BFS gir stien på kjøpet.
- **Syklus i urettet graf** → DFS med forelder-unntak. Det er standardløsningen, og den du blir spurt om.
- **Svært dyp graf** (lange kjeder) → iterativ DFS med `Stack<string>`, eller BFS. Rekursjon kan gå tom for stack.
- **Svært bred graf** (noder med tusenvis av naboer) → DFS bruker mindre minne enn BFS.
- **Vektete kanter** → ingen av dem. Da er det Dijkstra.

Samme skjelett, fire oppgaver

Alle fire funksjonene har samme kjerne: en beholder med noder som skal behandles, et besøkt-sett, og en løkke over naboer. Det som skiller dem er én linje hver: `parent[n] = cur` (korteste vei), `if (cur.Equals(goal)) return true` (rute), og `else if (!n.Equals(parent)) return true` (syklus). Pugg skjelettet én gang, og legg til linjen.

Del 6 Puggark

De fire funksjonene uten kommentarer og med korte navn. Dette er versjonen du skal kunne skrive på et blankt ark. Skriv ut denne siden.

Oppgave	Husk
Toveis forbindelse	AddNode nodeA, AddNode nodeB, nodeA→nodeB, nodeB→nodeA
BFS korteste vei	Kø, besøkt, forelder. Ny nabo: merk, forelder, kø. Mål: gå bakover, snu.
DFS rute	Mål? true. Merk. Ubesøkt nabo sier true? true. Ellers false.
DFS syklus	Merk. Ubesøkt: rekursjon, jeg er forelder. Besøkt og ikke forelder: true. Ytre løkke, rot = egen forelder.

```
// TOVEIS KANT
public void AddConnection(string nodeA, string nodeB)
{
    AddNode(nodeA); AddNode(nodeB);
    _adj[nodeA].Add(nodeB);
    _adj[nodeB].Add(nodeA);
}

// BFS: KORTESTE VEI
public List<string>? ShortestPath(string start, string goal)
{
    var queue = new Queue<string>();
    var visited = new HashSet<string> { start };
    var parent = new Dictionary<string, string>();
    queue.Enqueue(start);

    while (queue.Count > 0)
    {
        var cur = queue.Dequeue();
        if (cur.Equals(goal))
            return BuildPath(parent, start, goal);

        foreach (var n in Neighbors(cur))
            if (visited.Add(n))
            {
                parent[n] = cur;
                queue.Enqueue(n);
            }
    }
    return null;
}

static List<string> BuildPath(Dictionary<string, string> parent,
    string start, string goal)
{
    var path = new List<string>();
    for (var n = goal; !n.Equals(start); n = parent[n])
        path.Add(n);
    path.Add(start);
    path.Reverse();
    return path;
}
```

```
// DFS: FINNES RUTE?
public bool HasPath(string start, string goal)
    => Dfs(start, goal, new HashSet<string>());

bool Dfs(string cur, string goal, HashSet<string> visited)
{
    if (cur.Equals(goal)) return true;
    visited.Add(cur);
    foreach (var n in Neighbors(cur))
        if (!visited.Contains(n) && Dfs(n, goal, visited))
            return true;
    return false;
}

// DFS: SYKLUS I URETTET GRAF
public bool HasCycle()
{
    var visited = new HashSet<string>();
    foreach (var node in Nodes)
        if (!visited.Contains(node)
            && CycleDfs(node, node, visited))
            return true;
    return false;
}

bool CycleDfs(string cur, string parent, HashSet<string> visited)
{
    visited.Add(cur);
    foreach (var n in Neighbors(cur))
    {
        if (!visited.Contains(n))
        {
            if (CycleDfs(n, cur, visited)) return true;
        }
        else if (!n.Equals(parent))
            return true;
    }
    return false;
}
```

Del 7 Fyll inn hullene

Dekk til resten av dokumentet. Skriv det som mangler i hvert nummererte hull, og sjekk mot fasiten på neste side. Klarer du alle 16 uten å kikke, sitter det.

Toveis forbindelse

```
public void AddConnection(string nodeA, string nodeB)
{
    AddNode(nodeA);
    1;
    _adj[nodeA].Add(nodeB);
    2;
}
```

BFS: korteste vei

```
public List<string>? ShortestPath(string start, string goal)
{
    var queue = new Queue<string>();
    var visited = new HashSet<string> { 3 };
    var parent = new Dictionary<string, string>();
    queue.Enqueue(start);

    while ( 4 )
    {
        var current = queue.Dequeue();
        if (current.Equals(goal))
            return BuildPath(parent, start, goal);

        foreach (var neighbor in Neighbors(current))
        {
            if ( 5 )
            {
                6;
                queue.Enqueue(neighbor);
            }
        }
    }
    return 7;
}

private static List<string> BuildPath(Dictionary<string, string> parent, string start, string goal)
{
    var path = new List<string>();
    var node = goal;
    while ( 8 )
    {
        path.Add(node);
        node = 9;
    }
    path.Add(start);
    10;
    return path;
}
```

DFS: finnes det en rute?

```
private bool HasPathDfs(string current, string goal, HashSet<string> visited)
{
    if ( 11 )
        return true;

    visited.Add(current);

    foreach (var neighbor in Neighbors(current))
    {
        if (!visited.Contains(neighbor) && 12 )
            return true;
    }
    return 13;
}
```

DFS: syklus i urettet graf

```

public bool HasCycle()
{
    var visited = new HashSet<string>();
    foreach (var node in Nodes)
    {
        if (!visited.Contains(node) && CycleDfs(node, 14, visited))
            return true;
    }
    return false;
}

private bool CycleDfs(string current, string parent, HashSet<string> visited)
{
    visited.Add(current);
    foreach (var neighbor in Neighbors(current))
    {
        if (!visited.Contains(neighbor))
        {
            if (CycleDfs(neighbor, 15, visited))
                return true;
        }
        else if (16)
        {
            return true;
        }
    }
    return false;
}

```

Del 8 Test deg selv

Svar med eksempelgrafen i hodet. Fasit nederst på siden, sammen med svarene til fyll-inn-testen.

1. Hva returnerer `ShortestPath("A", "G")`, og hvorfor?
2. Spor `ShortestPath("D", "C")` for hånd. Hvilken sti får du, og finnes det en annen som er like kort?
3. Du fjerner linjen `_adj[nodeB].Add(nodeA)` fra `AddConnection`. Hva blir `HasPath("A", "F")` og `HasPath("F", "A")`?
4. I `CycleDfs` erstatter du `else if (!neighbor.Equals(parent))` med bare `else`. Hva blir `HasCycle()` på grafen uten C-E?
5. Hvorfor kan ikke `HasCycle()` nøye seg med å kalle `CycleDfs` fra én startnode?
6. Hvorfor må `visited` merkes når naboene legges i køen, og ikke når den tas ut?
7. Hva skjer i `HasPath("A", "F")` hvis du fjerner besøkt-settet?
8. Du bytter `Queue<string>` med `Stack<string>` i `ShortestPath`. Kompilerer det? Er svaret fortsatt korteste vei?

Fasit: spørsmål

1. `null`. Køen tømmes etter A, B, C, D, E, F uten at G blir oppdaget, fordi G ikke har noen kanter.
2. Kø: $D \rightarrow [B] \rightarrow [A, E] \rightarrow [E, C] \rightarrow [C, F]$. C tas ut som mål. Forelder: $B \leftarrow D$, $A \leftarrow B$, $E \leftarrow B$, $C \leftarrow A$, $F \leftarrow E$. Sti: $D \rightarrow B \rightarrow A \rightarrow C$, lengde 3. $D \rightarrow B \rightarrow E \rightarrow C$ er like kort, men A ble tatt ut av køen før E, så C ble oppdaget fra A.
3. Grafen er nå rettet, med kantene $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $B \rightarrow E$, $C \rightarrow E$, $E \rightarrow F$. `HasPath("A", "F")` er fortsatt `true` ($A \rightarrow B \rightarrow E \rightarrow F$), men `HasPath("F", "A")` er `false`: F sin naboliste er tom.
4. `true`, selv om grafen er en skog. Allerede i `CycleDfs(B, A)` ser B naboene A, som er besøkt, og uten unntaket meldes det som syklus. Enhver graf med én kant får samme svar.
5. Fordi grafen kan ha flere komponenter. En DFS fra A når aldri G, og hadde syklusen ligget der, hadde den vært usynlig.
6. Ellers kan samme node legges i køen flere ganger før den tas ut, og hver gang overskrives `parent`. Den siste som skrev kan ligge ett nivå lenger ute, og stien blir lengre enn nødvendig.
7. Uendelig rekursjon: $A \rightarrow B \rightarrow A \rightarrow B \dots$ til `StackOverflowException`. Alle kanter er toveis, så uten besøkt-settet finnes det alltid en vei tilbake.
8. Det kompilerer hvis du også bytter `Enqueue/Dequeue` med `Push/Pop`. Men da er det DFS, og stien er bare en vei, ikke den korteste. Fra A til F kan du for eksempel få $A \rightarrow C \rightarrow E \rightarrow B \rightarrow D \dots$ nei: du får $A \rightarrow C \rightarrow E \rightarrow F$, som tilfeldigvis er kortest her, men det er ikke garantert.

Fasit: fyll inn hullene

#	Svar	#	Svar
1	<code>AddNode(nodeB)</code>	9	<code>parent[node]</code>
2	<code>_adj[nodeB].Add(nodeA)</code>	10	<code>path.Reverse()</code>
3	<code>start</code>	11	<code>current.Equals(goal)</code>
4	<code>queue.Count > 0</code>	12	<code>HasPathDfs(neighbor, goal, visited)</code>
5	<code>visited.Add(neighbor)</code>	13	<code>false</code>
6	<code>parent[neighbor] = current</code>	14	<code>node</code>
7	<code>null</code>	15	<code>current</code>
8	<code>!node.Equals(start)</code>	16	<code>!neighbor.Equals(parent)</code>

Vedlegg Komplet program (Program.cs)

Alt i én fil. Lag et konsollprosjekt med `dotnet new console`, erstatt innholdet i `Program.cs` med blokkene under (i rekkefølge), og kjør `dotnet run`. Top-level statements må stå først i filen, klassen under. Forventet utskrift:

```
A -> B -> E -> F
True
3
True
False
True
```

```
using System;
using System.Collections.Generic;

var g = new Graph();

g.AddConnection("A", "B");
g.AddConnection("A", "C");
g.AddConnection("B", "D");
g.AddConnection("B", "E");
g.AddConnection("C", "E");
g.AddConnection("E", "F");
g.AddNode("G"); // isolert node

var path = g.ShortestPath("A", "F");
Console.WriteLine(string.Join(" -> ", path!)); // A -> B -> E -> F
Console.WriteLine(g.ShortestPath("A", "G") == null); // True
Console.WriteLine(g.Distances("A")["F"]); // 3

Console.WriteLine(g.HasPath("D", "F")); // True
Console.WriteLine(g.HasPath("A", "G")); // False

Console.WriteLine(g.HasCycle()); // True (A-B-E-C-A)
```

```
public class Graph
{
    private readonly Dictionary<string, List<string>> _adj = new();

    public IEnumerable<string> Nodes => _adj.Keys;

    public void AddNode(string node)
    {
        _adj.TryAdd(node, new List<string>());
    }

    public void AddConnection(string nodeA, string nodeB)
    {
        AddNode(nodeA);
        AddNode(nodeB);
        _adj[nodeA].Add(nodeB);
        _adj[nodeB].Add(nodeA);
    }

    public IReadOnlyList<string> Neighbors(string node)
    {
        if (_adj.TryGetValue(node, out var list))
            return list;
        return Array.Empty<string>();
    }
}
```

```
// ----- BFS -----

public List<string>? ShortestPath(string start, string goal)
{
    var queue = new Queue<string>();
    var visited = new HashSet<string> { start };
    var parent = new Dictionary<string, string>(); // barn -> forelder (hvem oppdaget hvem)

    queue.Enqueue(start);

    while (queue.Count > 0)
    {
        var current = queue.Dequeue();

        if (current.Equals(goal))
            return BuildPath(parent, start, goal);

        foreach (var neighbor in Neighbors(current))
        {
            if (visited.Add(neighbor)) // Add returnerer true bare første gang
            {
                parent[neighbor] = current; // <-- den ene ekstra linjen
                queue.Enqueue(neighbor);
            }
        }
    }

    return null; // goal kan ikke nås fra start
}

private static List<string> BuildPath(Dictionary<string, string> parent, string start, string goal)
{
    var path = new List<string>();
    var node = goal;

    while (!node.Equals(start)) // gå baklengs: mål -> ... -> start
    {
        path.Add(node);
        node = parent[node];
    }
    path.Add(start);
    path.Reverse(); // snu, så stien går start -> mål
    return path;
}

public Dictionary<string, int> Distances(string start)
{
    var dist = new Dictionary<string, int> { [start] = 0 }; // fungerer også som visited
    var queue = new Queue<string>();
    queue.Enqueue(start);

    while (queue.Count > 0)
    {
        var current = queue.Dequeue();
        foreach (var neighbor in Neighbors(current))
        {
            if (dist.TryAdd(neighbor, dist[current] + 1)) // ny node: ett steg lenger
                queue.Enqueue(neighbor);
        }
    }
    return dist; // dist[x] = antall steg fra start; mangler x, er den ikke nåbar
}

```

```
// ----- DFS -----

public bool HasPath(string start, string goal)
{
    return HasPathDfs(start, goal, new HashSet<string>());
}

private bool HasPathDfs(string current, string goal, HashSet<string> visited)
{
    if (current.Equals(goal))
        return true; // 1. står vi på målet?

    visited.Add(current); // 2. merk før vi går videre

    foreach (var neighbor in Neighbors(current))
    {
        if (!visited.Contains(neighbor) && HasPathDfs(neighbor, goal, visited))
            return true; // 3. funnet lenger nede -> send opp
    }

    return false; // 4. ingen av veiene førte fram
}

public bool HasPathIterative(string start, string goal)
{
    var stack = new Stack<string>();
    var visited = new HashSet<string>();
    stack.Push(start);

    while (stack.Count > 0)
    {
        var current = stack.Pop();
        if (current.Equals(goal)) return true;
        if (!visited.Add(current)) continue; // allerede behandlet

        foreach (var neighbor in Neighbors(current))
            if (!visited.Contains(neighbor))
                stack.Push(neighbor);
    }
    return false;
}
```

```
public bool HasCycle()
{
    var visited = new HashSet<string>();

    foreach (var node in Nodes) // ytre løkke: dekker alle komponenter
    {
        if (!visited.Contains(node) && CycleDfs(node, node, visited))
            return true; // roten er sin egen forelder
    }
    return false;
}

private bool CycleDfs(string current, string parent, HashSet<string> visited)
{
    visited.Add(current);

    foreach (var neighbor in Neighbors(current))
    {
        if (!visited.Contains(neighbor))
        {
            if (CycleDfs(neighbor, current, visited)) // jeg er nå forelder
                return true;
        }
        else if (!neighbor.Equals(parent))
        {
            return true; // besøkt nabo som IKKE er der vi kom fra = syklus
        }
    }
    return false;
}
}
```